PROOF OVER PROSE: PROGRAM-GENERATED FORMAL DERIVATIONS AS MIDTRAINING DATA

Anonymous authors
Paper under double-blind review

ABSTRACT

Frontier language models will exhaust the public supply of human text within the decade, and the usual remedy is to generate more text with a language model, which bounds the result by what the generating model already knows. We test a different source. A few hundred lines of Python sample a multi-hop deduction problem, solve it, and write the solution out, producing text that is correct by construction and needs no teacher. Replacing ten percent of a 2.5 billion token midtraining mixture for Qwen2.5-7B with these derivations raises accuracy on held-out ProofWriter problems from 44.4 to 57.0 percent at inference depth 3, where replacing the same share with ordinary long documents matched in length distribution leaves accuracy at the control level. Across a thousand generations the midtrained model never reproduces the generator's notation, so what transfers is the reasoning and not the format.

How the generator writes its solutions matters as well. Each latent proof is rendered twice, once as a formal derivation with numbered lines and named inference rules and once as a controlled English explanation, holding the prompt, the answer and the proof graph fixed so that only the notation differs. Which rendering trains the better reasoner depends on how many hops the training proofs contain. English generalizes further through five to twenty hops, and at twenty-five the ordering reverses, with formal supervision at 75.0 ± 9.5 percent out-of-distribution accuracy against 17.8 ± 4.7. Formal supervision also reaches 99.6 ± 0.3 percent within sixteen samples where English reaches 42.5 ± 5.8, and because a formal derivation is mechanically checkable that breadth converts into 97.3 ± 1.1 percent single-answer accuracy with no answer key.

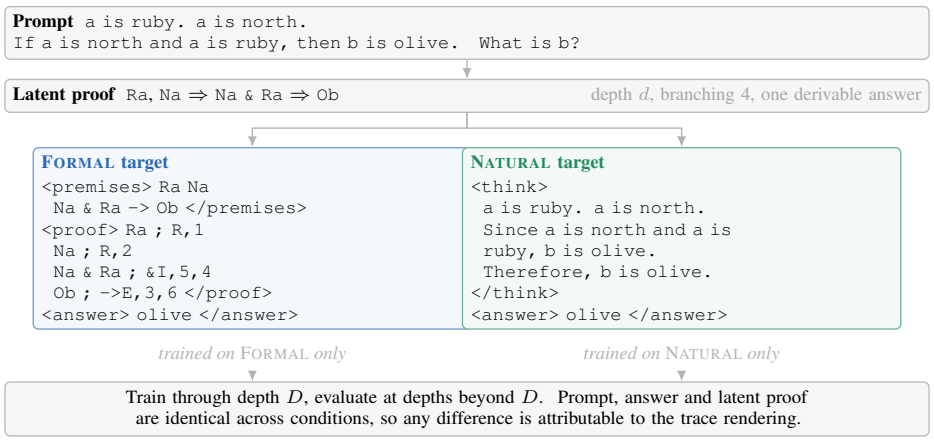

Figure 1: How the two supervised targets are produced. A generator samples a latent proof and writes it out as a formal derivation or as a controlled English explanation. The prompt, the answer and the proof graph are shared, and only the notation of the supervised target differs. Models are trained on one notation and evaluated beyond the training depth range.

1 INTRODUCTION

Public human text is a finite resource, and the point at which frontier training runs exhaust it is now close enough to plan around (Villalobos et al., 2024). The usual answer is synthetic data, and almost every version of that answer asks a language model to write it. Textbook-style corpora are generated by a stronger model (Gunasekar et al., 2023), web documents are rewritten by an instruction-tuned one (Maini et al., 2024; Pieler et al., 2025), and a small corpus is expanded by prompting (Yang et al., 2025). All of these inherit whatever the generating model can already do, and they carry the risk of degradation when a model is trained on its own distribution.

There is another source of synthetic text that has no model in it at all. A few hundred lines of Python can sample a multi-hop deduction problem, solve it, and write the solution out, and the result is correct by construction, unlimited in quantity, and free of any teacher's blind spots. Whether text like that is worth putting into a real training mixture is an open question, since it is narrow, unnatural, and nothing like the web.

We put it in. Replacing ten percent of a 2.5 billion token midtraining mixture for Qwen2.5-7B with such derivations raises accuracy on held-out ProofWriter problems (Tafjord et al., 2021) that the generator never produced, from 44.4 to 57.0 percent at inference depth 3. Replacing the same ten percent with ordinary long documents from the same corpus, matched to the derivations in length distribution, leaves accuracy at the control level, so the gain follows from the deduction the documents contain. The model does not learn to imitate the generator. Not one of its thousand greedy generations reproduces the notation it was trained on. What survives instruction tuning is the skill, not the format.

How the generator writes its solutions turns out to matter as much as what it generates. The same latent proof can be written as a formal derivation, with numbered lines and named inference rules, or as a controlled English explanation. We generate both from one source, so that the prompt, the answer and the proof graph are identical within a pair and only the notation differs (Figure 1). Two models trained on opposite sides of that pair are as close to a controlled comparison as training data allows.

Formal supervision wins, but only once the training proofs are deep enough. At five to twenty hops the English rendering generalizes further to unseen depths. At twenty-five hops the ordering reverses by a factor of four, with formal supervision at 75.0 ± 9.5 percent out-of-distribution accuracy against 17.8 ± 4.7 (Table 1). Neither target length nor the number of supervised tokens accounts for the reversal, and an independently generated constraint-propagation task reproduces it. Drawing sixteen samples instead of one moves the reversal ten hops earlier, so formal supervision widens the set of problems a model can solve before it improves the model's first attempt. Because a formal derivation is mechanically checkable, that wider set is usable without an answer key, and returning the first sample a proof checker accepts reaches 97.3 ± 1.1 percent against 33.8 ± 1.4 for English traces.

Our reading of this is that programmatic generators deserve a place in the midtraining mixture, and that the notation they emit is a design parameter and not a formatting choice. A generator costs almost nothing to run, targets a named capability, and needs no teacher model. If deduction responds this way, other capabilities with cheap programmatic generators are worth testing the same way, which is a different route around the data wall from asking a larger model to write more text.

2 RELATED WORK

Synthetic data for pretraining. Projections of the public text supply place its exhaustion within the current decade (Villalobos et al., 2024), and synthetic generation is the main proposed remedy. Small models trained on generated textbook-style corpora reach far above their weight class (Gunasekar et al., 2023), rewriting web documents into cleaner prose accelerates pretraining several-fold (Maini et al., 2024), the recipe has been scaled to trillions of tokens (Pieler et al., 2025), and a small domain corpus can be expanded by prompting before continued pretraining (Yang et al., 2025). Every one of these uses a language model as the generator, so the resulting corpus is bounded by what that model already knows and the pipeline needs an expensive teacher. Our generator is a program. It has no teacher, it cannot hallucinate a step, and its output is verifiable by the same code that produced it.

Reasoning traces. Writing out intermediate steps improves multi-step accuracy (Wei et al., 2022; Kojima et al., 2022), and the idea has been extended by bootstrapping rationales (Zelikman et al., 2022), decomposing questions (Zhou et al., 2023), searching over partial states (Yao et al., 2023) and supervising a scratchpad (Nye et al., 2021), with gains concentrated in mathematical and symbolic domains (Sprague et al., 2025). Theory accounts for part of this, since intermediate tokens let a constant-depth transformer carry out serial computation it cannot perform in one forward pass (Feng et al., 2023; Merrill & Sabharwal, 2024; Li et al., 2024b). This literature writes its steps in prose without examining that choice, which is the variable we manipulate.

Formal systems. Formal notation is normally something a model produces for a checker. Interactive provers such as Lean, Isabelle and Coq have been targeted directly (Polu & Sutskever, 2020; Jiang et al., 2023; Yang et al., 2023; Song et al., 2024), a literature surveyed at length (Li et al., 2024a), autoformalization translates informal statements into those systems (Wu et al., 2022; Azerbayev et al., 2023), program-aided prompting delegates arithmetic to an interpreter (Gao et al., 2023; Chen et al., 2023), logic pipelines hand problems to a solver (Pan et al., 2023; Ling et al., 2023), and traces can be typed to make them verifiable (Perrier, 2025). In each case the formal object earns its keep at inference time through an external tool. We use formality only as a property of the training corpus. No prover runs at inference, and the midtrained models stop emitting formal notation entirely while keeping the benefit, which suggests the two uses of formality are largely independent.

Controlled study of what models learn. Synthetic corpora with known generative structure support questions that benchmark accuracy cannot settle, as in the iGSM family (Ye et al., 2025a;b) and in work controlling the generating grammar (Allen-Zhu & Li, 2023). Models trained to reason can satisfy the objective through statistical shortcuts (Zhang et al., 2023), fail on compositions that scale does not repair (Dziri et al., 2023), and show error modes that aggregate scores conceal (Saparov & He, 2023; Clark et al., 2020; Tafjord et al., 2021), while the locality of the training distribution has been offered as the reason intermediate steps help at all (Prystawski et al., 2023). Rationales can also omit what actually drove a prediction (Turpin et al., 2023; Lanham et al., 2023), so we report answer accuracy and proof validity separately. Our depth split follows the usual practice of controlling train and test distributions (Lake & Baroni, 2018; Keysers et al., 2020; Hupkes et al., 2020; Anil et al., 2022), with a spurious-marker variant to test shortcut reliance (Geirhos et al., 2020).

Sampling and verification. Repeated sampling converts inference compute into solved problems (Brown et al., 2024), verifiers and process supervision select among the samples (Cobbe et al., 2021; Lightman et al., 2024), self-consistency selects by agreement (Wang et al., 2023), and how to spend a fixed budget has been studied empirically (Snell et al., 2025; Muennighoff et al., 2025) and theoretically (Setlur et al., 2025). One recent argument holds that verifiable-reward training mostly sharpens a model toward what it could already sample (Yue et al., 2025; Guo et al., 2025; Shao et al., 2024; Gandhi et al., 2025). Our checker-based selection result belongs to the controlled setting only, for the reason given above, so we raise the connection in Section 7 and do not claim it at midtraining scale.

3 Data and Experimental Setup

Each example begins with a latent directed acyclic proof. A generator emits a prompt containing ground facts, rules, a query and distractor branches, then traverses the supporting path to produce the answer. A rendering function writes that path in one of two notations. The Formal rendering declares constants and predicates and applies named inference rules to numbered lines. The Natural rendering expresses the same substitutions and conclusions, line for line, in controlled sentences. Both targets end with the same answer field and the prompt is natural language in both conditions.

BranchProof composes Horn-style inferences in which every step offers four locally plausible branches, exactly one derivable from the current state, with a fresh constant introduced at every layer. The generator computes the full Horn closure of each example and accepts it only when exactly one candidate answer is derivable, which removes any reward for guessing among defensible answers. AttrCon propagates values among object attributes under generated constraints, and its vocabulary and proof construction are disjoint from BranchProof. Generation parameters are held fixed across every controlled experiment, at branching factor 4 and distractor ratio 0.5, with depths sampled

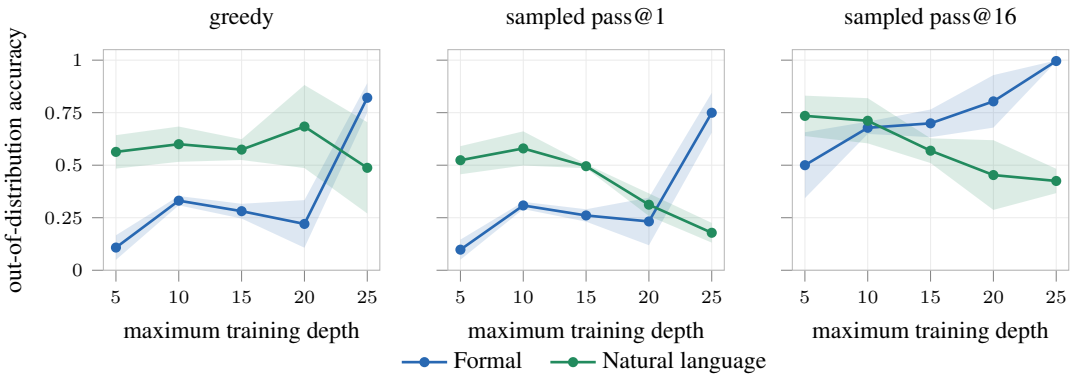

Figure 2: Out-of-distribution answer accuracy on BRANCHPROOF against maximum training depth, for three decoding rules. Shaded regions give one standard deviation over three seeds. Greedy decoding and pass@1 place the crossover at maximum training depth 25. Sampling with sixteen draws places it at 15, ten depth levels earlier.

Table 1: Out-of-distribution answer accuracy on BRANCHPROOF, in percent, mean and standard deviation over three seeds. Greedy decoding and pass@1 order the conditions identically. pass@16 reverses ten hops earlier.

	maximum training depth				
metric and condition	5	10	15	20	25
greedy, FORMAL	10.8 ± 5.9	33.1 ± 2.2	28.1 ± 3.5	22.1 ± 11.3	82.1 ± 7.0
greedy, NATURAL	56.3 ± 8.0	60.0 ± 8.4	57.4 ± 5.0	68.4 ± 19.8	48.8 ± 21.7
pass@1, FORMAL	9.8 ± 4.8	30.8 ± 1.7	26.1 ± 2.9	23.2 ± 11.3	75.0 ± 9.5
pass@1, NATURAL	52.4 ± 6.7	58.0 ± 8.1	49.5 ± 1.0	31.2 ± 5.4	17.8 ± 4.7
pass@16, FORMAL	50.0 ± 15.7	67.8 ± 2.9	69.9 ± 6.6	80.4 ± 12.5	99.6 ± 0.3
pass@16, NATURAL	73.5 ± 9.6	71.2 ± 10.8	56.9 ± 5.9	45.3 ± 16.6	42.5 ± 5.8

round-robin. Corpus seeds are disjoint across training, evaluation and sampling roles. Appendices G to F give paired targets, the grammar, the generator and the validators, and Appendix A reports the answer-uniqueness validation.

The controlled experiment fine-tunes OLMo-3-7B (OLMo Team et al., 2025) with LoRA on 50,000 materialized examples per condition, with maximum training depth in $\{5, 10, 15, 20, 25\}$ and three seeds per condition. Evaluation covers depths $\{1, 2, 5, 10, 12, 15, 18, 20, 25, 30, 35, 40, 45, 50\}$ with 32 prompts per depth, and at each prompt sixteen samples at temperature 1.0 together with one greedy decode. Sampled coverage uses the unbiased pass@k estimator (Chen et al., 2021). Both notations receive an identical 7,168-token generation limit inside a 16,384-token window, and a tokenizer audit confirms every gold prompt and target fits in both renderings, so neither condition is truncated. Depths above the maximum training depth are out of distribution. Joint metrics require the correct answer together with a mechanically valid proof, where formal traces are checked directly by the proof engine and natural traces by a deterministic translation into the same engine, so answer-only accuracy is reported alongside every joint metric. Appendices D and H give the remaining configuration and the metric definitions.

4 HOW PROOF DEPTH CHANGES THE COMPARISON

Out-of-distribution answer accuracy as a function of maximum training depth is given in Table 1. Through maximum training depth 20 the natural condition is ahead on every decoding rule. At maximum training depth 25 the formal condition reaches 0.750 pass@1 against 0.178, a factor of four, and 0.996 pass@16 against 0.425.

The two decoding rules do not place the reversal at the same depth. Greedy accuracy and pass@1 both cross at maximum training depth 25, while pass@16 crosses at maximum training depth 15, where the formal condition reaches 69.9 percent against 56.9. Between the two crossings lies a regime in which formal supervision has produced a model that can construct a correct derivation within sixteen attempts but not on the first, so the set of solvable problems widens before single-sample reliability improves. A study reporting greedy accuracy alone would locate the effect ten hops too late and describe that range as showing no formal advantage.

English degrades monotonically in pass@16 from 73.5 percent at five hops to 42.5 at twenty-five, while formal supervision rises monotonically from 50.0 to 99.6, so the two notations respond to deeper training proofs in opposite directions. Neither is better in general. Which one produces the stronger reasoner flips as the training proofs get deeper, and where it flips depends on how the model is decoded, which makes the decoding rule part of the experimental design. The two readings measure different things, and the sampled one is what matters for any system that draws more than one sample.

Length and budget controls. Formal and natural targets differ in length, which raises the possibility that the comparison measures supervision quantity. Two controls address this. Token budgets at generation are identical by construction, and the tokenizer audit confirms no truncation in either condition. Matched-token training, in which the number of supervised target tokens is equated across conditions, leaves the ordering unchanged. Details are in Appendix I.

Replication on an independent task. ATTRCON shares no vocabulary and no proof construction with BRANCHPROOF. The correctness advantage of formal supervision at high training depth reproduces there, which makes it unlikely that the effect depends on a particular rule system.

4.1 CHECKING SAMPLES WITHOUT AN ANSWER KEY

A model whose sixteen samples contain a correct proof is only useful if that proof can be picked out without already knowing the answer. For formal derivations it can be, since validity is decidable by the same engine that generated the problem. Drawing sixteen samples, checking each in order and returning the answer of the first one the checker accepts yields 97.3 ± 1.1 percent single-answer accuracy at maximum training depth 25, against a joint pass@16 ceiling of 98.1 ± 0.5 percent. The 0.8 point gap is the cost of derivations the checker accepts that reach a wrong answer, so selection is close to the ceiling that coverage permits. No reward model and no gold labels enter the procedure.

The identical selector applied to translated English traces recovers 33.8 ± 1.4 percent. The deficit is not primarily a failure of translation. Long English derivations frequently fail to terminate inside the generation budget, so no checkable derivation exists to accept. This holds for models trained on formal derivations alone, and Section 6.5 shows that it does not carry over to the midtraining setting, where the models stop emitting formal derivations altogether.

5 CONTROLS

Table 2 reports controls run on the same corrected generator as Table 1, so the entries are directly comparable to the depth-25 row there.

Equating the number of supervised target tokens across conditions leaves the ordering intact, at 69.8 ± 34.9 pass@16 for the token-matched English condition against 99.6 ± 0.3 for formal supervision, so the gap is not an effect of supervision quantity. The variance of that control is very large, which is worth noting alongside the point estimate.

A spurious marker planted in training and removed at test time reverses the comparison. English supervision is barely affected, at 99.4 ± 0.9 pass@16 and 76.1 percent at depth 50, while formal supervision falls to 87.3 ± 7.5 and to 11.7 percent at depth 50. The formal advantage is therefore not generic robustness to distribution shift, and a model trained on rigid syntax appears to bind more readily to a rigid spurious feature.

Putting both notations in one model fails when they are concatenated into a single target, at 0.5 ± 0.4 and 0.2 ± 0.1 pass@1, because the model copies one or both long traces and exhausts its budget.

Table 2: Out-of-distribution answer accuracy in percent, mean and standard deviation over three seeds, for controls run on the same corrected generator as Table 1. All entries train through depth 25.

condition	pass@1	pass@16
reference, maximum training depth 25		
formal derivation	75.0 ± 9.5	99.6 ± 0.3
English explanation	17.8 ± 4.7	42.5 ± 5.8
supervision-quantity control		
English, target tokens matched to formal	41.0 ± 20.1	69.8 ± 34.9
spurious marker planted in training, absent at test		
formal	43.1 ± 7.7	87.3 ± 7.5
English	78.5 ± 14.9	99.4 ± 0.9
both notations in one model		
concatenated, formal first	0.5 ± 0.4	7.1 ± 5.5
concatenated, English first	0.2 ± 0.1	2.3 ± 1.6
mode-conditioned, formal mode	50.3 ± 6.4	87.1 ± 3.9
mode-conditioned, English mode	18.1 ± 2.0	24.8 ± 3.7
other base models		
Qwen2.5-7B, formal	69.9 ± 11.1	77.3 ± 5.2
Qwen2.5-7B, English	74.9 ± 7.0	76.0 ± 5.6
OLMo-3-32B, formal	63.9 ± 5.1	98.8 ± 0.9
OLMo-3-32B, English	98.9 ± 0.8	99.8 ± 0.3
OLMo-3-32B conditioned, formal mode	85.0 ± 8.7	99.8 ± 0.3

Mode-conditioned training, with an output-mode tag on separate examples, works better and reaches 50.3 ± 6.4 pass@1 in formal mode.

Capacity reorders the comparison. On Qwen2.5-7B the conditions are close, at 69.9 ± 11.1 against 74.9 ± 7.0 pass@1. On OLMo-3-32B English leads decisively at 98.9 ± 0.8 against 63.9 ± 5.1, while formal coverage stays near ceiling at 98.8 ± 0.9 pass@16, so the deficit at 32B is one of probability mass, since coverage is intact. Mode conditioning at 32B lifts the formal mode to 85.0 ± 8.7 pass@1, still below single-modality English. Models trained from scratch at 50M to 200M parameters fail in both conditions at every size, so the effect operates on competence supplied by pretraining.

6 Midtraining a 7B Model on Formal Derivations

Whether the same variable matters when derivations are a small minority of an ordinary training mixture is tested by midtraining Qwen2.5-7B (Yang et al., 2024) on a production corpus with a fixed fraction replaced by matched derivations, on a different model family and against benchmarks the generator never saw.

6.1 Design

Five conditions share every hyperparameter, every data path outside the replaced slice, and an identical token budget of 2.5 billion training tokens, reached as 2,385 steps of global batch 128 at sequence length 8,192. The conditions differ only in the ten percent of loss tokens that is replaced.

1. **Control** replaces nothing and trains on the unmodified mixture.
2. **LongDoc** replaces ten percent with long documents drawn from the same production corpus, selected to match the token-length histogram of the formal traces and carrying no deep deduction.
3. **Logic** replaces ten percent with formal derivations at depth 25.
4. **Natural** replaces ten percent with the natural-language rendering of the same latent proofs.
5. **Condensed** replaces ten percent with a compact formal rendering of the same latent proofs.

Table 3: Greedy exact match on the external ProofWriter items by inference depth, in percent, 500 items per depth, one training run per condition. LongDoc is length-matched to the formal traces and carries no deep deduction. Depth 0 requires lookup and no inference. Greedy decoding favours the English condition here, and Table 4 gives the sampled comparison.

inference depth	0	1	2	3	5
Control	44.2	32.8	44.0	44.4	46.0
LongDoc	45.8	32.2	44.4	45.4	44.0
Logic	48.0	35.8	49.4	50.0	50.2
Natural	48.8	38.8	56.2	57.0	56.4
Condensed	46.0	34.6	46.0	48.0	47.4

LongDoc separates two explanations of any gain, since formal derivations are long and long documents change what a model learns about maintaining state. Matching the length histogram while removing the deduction isolates the second factor. Median length is $3{,}712$ tokens against $3{,}750$ for the formal traces, and all thirty populated 250-token bins agree within about three percent.

6.2 Corpora and packing

A corpus of 72,000 examples is generated with depths 1 to 25 round-robin and the same generator settings as the controlled study. The three trace corpora render the same latent proofs, so the comparison between them holds the underlying computation fixed. Rendered lengths and packing statistics are in Appendix K.

A document longer than one window is excluded and counted, and none is split across windows, so the mid-derivation truncation that would destroy the depth structure under study cannot occur. Zero documents were excluded and zero split in every condition. Padding uses a dedicated token masked out of the loss. A decoded-batch audit verifies both properties on real loader windows and training refuses to start if it fails.

Because the replaced slice is fixed in tokens and the condensed rendering is 2.6 times shorter, the condensed condition traverses its proof set 2.31 times where Logic and Natural traverse theirs 0.90 and 0.89 times. Any comparison involving the condensed condition therefore confounds rendering with exposure, and we treat it accordingly in Section 6.4.

6.3 Instruction tuning and evaluation

All five checkpoints receive an identical instruction-tuning stage. Evaluation uses two families. The external family is ProofWriter (Tafjord et al., 2021) under the open-world assumption at inference depths 0, 1, 2, 3 and 5 with 500 items per depth, which the generator never saw. The near-transfer family asks for a derivation on held-out problems from the same generator at depths 5 to 25 with 200 items per depth. The metric is exact match. Sampled measurements draw sixteen completions and replay the greedy prompts, so the two decodings differ in the sampler alone. Appendix K gives every setting.

6.4 Results

On the external ProofWriter items both trace conditions improve over Control and over LongDoc, and LongDoc does not improve over Control (Table 3). At inference depth 0, which requires lookup and no inference, all five conditions fall between 0.442 and 0.488, and the separation appears only once inference is required. That is the shape a deduction effect should have, and it is not reproduced by exposure to long documents alone.

The near-transfer family separates the conditions far more sharply. Control sits near the floor at 0.000 to 0.040 exact match across depths, where every condition trained on derivations reaches 0.23 to 0.29, so the replaced slice teaches a capability the control mixture does not supply at all.

Under greedy decoding the natural condition is ahead of the formal condition on thirteen of fifteen tasks. Under sampling the ordering reverses. Formal minus English moves from -17.5, -16.5,

Table 4: Differences in pass@16, in percentage points, pooled over inference depths 5 to 25, with a standard deviation from a paired bootstrap over evaluation documents. The bootstrap covers evaluation sampling error only, and Section 6.6 treats training-run variance separately.

contrast	difference
Condensed − Control	$+27.6 \pm 1.9$
Logic − Control	$+22.6 \pm 2.0$
English − Control	$+17.6 \pm 2.0$
LongDoc − Control	$+13.8 \pm 1.7$
Condensed − English	$+10.0 \pm 1.7$
Logic − LongDoc	$+8.8 \pm 1.9$
Logic − English	$+5.0 \pm 1.9$

-5.5, -4.0 and -10.5 points under greedy decoding at inference depths 5 through 25 to $+3.5$, $+7.0$, $+14.0$, $+4.0$ and -3.5 at pass@16. Under majority voting over the same sixteen samples the formal condition does not lead, which is what the mechanism predicts, since majority voting rewards a concentrated mode where the checker rewards breadth. Appendix M gives the full table.

Pooling over depths and bootstrapping over documents with 4,000 resamples gives the contrasts in Table 4. Both trace conditions and LongDoc separate from Control, the trace conditions separate from LongDoc, and the formal condition exceeds the natural condition by 0.050 in pass@16 and by 0.025 in mean sampled accuracy.

Formal supervision is behind under greedy decoding and ahead under sampling, the same pattern that separated the two crossover depths in Section 4, now on a different model family with the derivations forming a tenth of an ordinary mixture. The proportion of greedy generations reaching a parseable answer is 0.645 to 0.785 for Control and 0.940 to 1.000 for the trace conditions, so the conditions differ in whether a derivation is completed at all.

The condensed condition attains the largest gain over Control and exceeds the formal condition by 0.050. Its documents are 2.6 times shorter, so its advantage is not a length effect. It traverses its proof set 2.6 times more often, so its advantage is confounded with exposure and cannot be attributed to the rendering. Resolving that requires an exposure-matched condensed corpus, which is not part of this work.

6.5 The midtrained models do not write formal proofs

None of the 1,000 greedy generations of the formal arm contains any marker of the training rendering, and the same holds for every other condition, so after instruction tuning all five models answer in prose. The gain reported above therefore cannot be an effect of the model adopting formal notation, and what the formal slice contributes is exposure to explicit and uniformly structured derivations, which survives instruction tuning while the notation does not. The selection result of Section 4.1 correspondingly does not apply at this scale, since no formal derivation is emitted for a checker to accept.

6.6 How far this result goes

Every midtraining condition was trained once, so the intervals in Table 4 estimate evaluation sampling error alone. An auxiliary experiment on the same suite, in Appendix B, puts the per-run standard deviation near 0.019, and therefore near 0.027 for a difference between two independently seeded conditions. Combined with the evaluation term that gives about 0.033 for the formal against English contrast, whose interval then becomes approximately $[-0.015, +0.115]$ and includes zero.

The contrasts against Control, between 0.176 and 0.276, are six to ten times the training standard deviation and are not at risk from single-seed training. The contrast between the two notations, at 0.050, is roughly twice that standard deviation and is a sign reversal against the greedy ordering, which is the pattern most likely to be produced by run-to-run variation. We therefore report the transfer-scale formal advantage as consistent with the controlled study. We do not claim it is independently established. Replication under a second instruction-tuning seed is the appropriate

remedy, since enlarging the evaluation set would shrink the smaller error term and leave the larger untouched.

The depth of the replaced derivations was fixed at 25 in every condition, so the transfer study locates no threshold of its own and inherits the one estimated from OLMo-3-7B in Section 4. An earlier configuration at sequence length 4,096, with a five percent replacement share and derivations capped at depth 14, produced no measurable transfer. It differs from the present configuration in window, replacement share, rendering and evaluation suite, so it motivates this design and is not a controlled below-threshold comparison.

There is a connection here to verifiable-reward posttraining. If such training principally reweights toward completions the base model can already produce (Yue et al., 2025), then the breadth a model has after midtraining bounds what the later stage reaches, and the notation moves that breadth by between 5.0 and 27.6 points of pass@16 here. Verifying the ordering requires running that stage on both conditions, which we have not done.

7 Limitations and Outlook

The controlled study uses one model family at 7B and one at 32B, and the transfer study uses a third. Depth thresholds estimated on OLMo-3-7B are applied to Qwen2.5-7B by assumption. We did not measure them there.

The transfer conditions were trained once each, with the consequences set out in Section 6.6.

Proof validity for natural-language traces is established through a deterministic translation into the proof engine. The translation is a component that the formal condition does not require, and although answer-only accuracy is reported alongside every joint metric, the joint natural-language numbers carry that dependency.

The generator produces one family of Horn-style deduction problems and one constraint-propagation family. Neither involves arithmetic, natural-language ambiguity or retrieval, and the depth axis studied here is proof depth. Context length and problem breadth are untouched.

The condensed rendering is confounded with exposure, as described in Section 6.4.

Two properties of formal supervision showed up in the controlled study that the midtraining experiment cannot yet use. Trained through twenty-five hops, the formal condition reaches 99.6 ± 0.3 percent pass@16 where English reaches 42.5 ± 5.8, so almost every held-out problem has a correct derivation somewhere in sixteen samples even at depths well beyond the training range, and every one of those problems carries locally plausible distractor branches by construction. That coverage is also harvestable, since the derivation is checkable without the answer, which converts sixteen samples into 97.3 ± 1.1 percent single-answer accuracy with no labels and no reward model. A system that samples and verifies wants exactly these two properties.

Neither is available after midtraining, because a tenth of the mixture is not enough to make the model write derivations it can be checked on. The natural next step is therefore to push much harder in this direction instead of treating ten percent as a ceiling. A larger synthetic share, generators that cover more of the reasoning the model will be asked to do, or an instruction stage that preserves the notation instead of washing it out, would each test whether a model can be brought to the point of actually reasoning in a checkable formalism while keeping the benefits reported here. Our results do not establish that this is achievable, and they do suggest it is worth the attempt, since the two properties above are what a verifier-in-the-loop system needs and prose supervision supplies neither.

The wider question is how far programmatic generators go. Deduction happens to be easy to generate and easy to check, which is why we chose it, and other capabilities with cheap generators, such as state tracking, constraint satisfaction, unit conversion or program tracing, could be tested the same way. If several of them behave as deduction does here, the practical consequence is that a midtraining mixture can be assembled partly from programs targeting named skills, at a cost that does not scale with the size of a teacher model.

8 Conclusion

Holding a derivation fixed and changing only the language in which it is written changes what a model learns from it, and the direction is set by the depth of the derivations in the training range. Formal supervision acquires coverage before single-sample accuracy, placing two thresholds ten depth levels apart and making the decoding rule part of the experimental design. Because formal derivations are checkable, that coverage converts into single-answer accuracy without labels, at 97.3 percent against a 98.1 percent ceiling. The same dissociation appears when matched derivations form a tenth of an ordinary midtraining mixture, against both an untreated control and a length-matched control without deduction.

References

Zeyuan Allen-Zhu and Yuanzhi Li. Physics of language models: Part 1, learning hierarchical language structures. *arXiv preprint arXiv:2305.13673*, 2023. doi: 10.48550/arXiv.2305.13673.

Cem Anil, Yuhuai Wu, Anders Andreassen, Aitor Lewkowycz, Vedant Misra, Vinay Ramasesh, Ambrose Slone, Guy Gur-Ari, Ethan Dyer, and Behnam Neyshabur. Exploring length generalization in large language models. In *Advances in Neural Information Processing Systems 35*, 2022.

Zhangir Azerbayev, Bartosz Piotrowski, Hailey Schoelkopf, Edward W. Ayers, Dragomir Radev, and Jeremy Avigad. ProofNet: Autoformalizing and formally proving undergraduate-level mathematics. *arXiv preprint arXiv:2302.12433*, 2023. doi: 10.48550/arXiv.2302.12433.

Bradley Brown, Jordan Juravsky, Ryan Ehrlich, Ronald Clark, Quoc V. Le, Christopher Ré, and Azalia Mirhoseini. Large language monkeys: Scaling inference compute with repeated sampling. *arXiv preprint arXiv:2407.21787*, 2024. doi: 10.48550/arXiv.2407.21787.

Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021. doi: 10.48550/arXiv.2107.03374.

Wenhu Chen, Xueguang Ma, Xinyi Wang, and William W. Cohen. Program of thoughts prompting: Disentangling computation from reasoning for numerical reasoning tasks. *Transactions on Machine Learning Research*, 2023.

Peter Clark, Oyvind Tafjord, and Kyle Richardson. Transformers as soft reasoners over language. In *Proceedings of the 29th International Joint Conference on Artificial Intelligence*, 2020.

Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021. doi: 10.48550/arXiv.2110.14168.

Nouha Dziri, Ximing Lu, Melanie Sclar, Xiang Lorraine Li, Liwei Jiang, Bill Yuchen Lin, Sean Welleck, Peter West, Chandra Bhagavatula, Ronan Le Bras, et al. Faith and fate: Limits of transformers on compositionality. In *Advances in Neural Information Processing Systems 36*, 2023.

Guhao Feng, Bohang Zhang, Yuntian Gu, Haotian Ye, Di He, and Liwei Wang. Towards revealing the mystery behind chain of thought: A theoretical perspective. In *Advances in Neural Information Processing Systems 36*, 2023.

Kanishk Gandhi, Ayush Chakravarthy, Anikait Singh, Nathan Lile, and Noah D. Goodman. Cognitive behaviors that enable self-improving reasoners, or, four habits of highly effective STaRs. In *Conference on Language Modeling*, 2025. doi: 10.48550/arXiv.2503.01307.

Luyu Gao, Aman Madaan, Shuyan Zhou, Uri Alon, Pengfei Liu, Yiming Yang, Jamie Callan, and Graham Neubig. PAL: Program-aided language models. In *Proceedings of the 40th International Conference on Machine Learning*, 2023.

Robert Geirhos, Jörn-Henrik Jacobsen, Claudio Michaelis, Richard Zemel, Wieland Brendel, Matthias Bethge, and Felix A. Wichmann. Shortcut learning in deep neural networks. *Nature Machine Intelligence*, 2(11):665–673, 2020. doi: 10.1038/s42256-020-00257-z.

Suriya Gunasekar, Yi Zhang, Jyoti Aneja, Caio César Teodoro Mendes, Allie Del Giorno, Sivakanth Gopi, Mojan Javaheripi, Piero Kauffmann, Gustavo de Rosa, Olli Saarikivi, et al. Textbooks are all you need. *arXiv preprint arXiv:2306.11644*, 2023. doi: 10.48550/arXiv.2306.11644.

Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Ruoyu Zhang, Runxin Xu, Qihao Zhu, Shirong Ma, Peiyi Wang, Xiao Bi, et al. DeepSeek-R1 incentivizes reasoning in LLMs through reinforcement learning. *Nature*, 645(8081):633–638, 2025. doi: 10.1038/s41586-025-09422-z.

Dieuwke Hupkes, Verna Dankers, Mathijs Mul, and Elia Bruni. Compositionality decomposed: How do neural networks generalise? *Journal of Artificial Intelligence Research*, 67:757–795, 2020. doi: 10.1613/jair.1.11674.

Albert Q. Jiang, Sean Welleck, Jin Peng Zhou, Wenda Li, Jiacheng Liu, Mateja Jamnik, Timothée Lacroix, Yuhuai Wu, and Guillaume Lample. Draft, sketch, and prove: Guiding formal theorem provers with informal proofs. In *International Conference on Learning Representations*, 2023. doi: 10.48550/arXiv.2210.12283.

Daniel Keysers, Nathanael Schärli, Nathan Scales, Hylke Buisman, Daniel Furrer, Sergii Kashubin, Nikola Momchev, Danila Sinopalnikov, Lukasz Stafiniak, Tibor Tihon, Dmitry Tsarkov, Xiao Wang, Marc van Zee, and Olivier Bousquet. Measuring compositional generalization: A comprehensive method on realistic data. In *International Conference on Learning Representations*, 2020.

Takeshi Kojima, Shixiang Shane Gu, Machel Reid, Yutaka Matsuo, and Yusuke Iwasawa. Large language models are zero-shot reasoners. In *Advances in Neural Information Processing Systems 35*, 2022. doi: 10.48550/arXiv.2205.11916.

Brenden M. Lake and Marco Baroni. Generalization without systematicity: On the compositional skills of sequence-to-sequence recurrent networks. In *Proceedings of the 35th International Conference on Machine Learning*, 2018.

Tamera Lanham, Anna Chen, Ansh Radhakrishnan, Benoit Steiner, Carson Denison, Danny Hernandez, Dustin Li, Esin Durmus, Evan Hubinger, Jackson Kernion, et al. Measuring faithfulness in chain-of-thought reasoning. *arXiv preprint arXiv:2307.13702*, 2023. doi: 10.48550/arXiv.2307.13702.

Zhaoyu Li, Jialiang Sun, Logan Murphy, Qidong Su, Zenan Li, Xian Zhang, Kaiyu Yang, and Xujie Si. A survey on deep learning for theorem proving. *Conference on Language Modeling*, 2024a. doi: 10.48550/arXiv.2404.09939.

Zhiyuan Li, Hong Liu, Denny Zhou, and Tengyu Ma. Chain of thought empowers transformers to solve inherently serial problems. In *International Conference on Learning Representations*, 2024b. doi: 10.48550/arXiv.2402.12875.

Hunter Lightman, Vineet Kosaraju, Yuri Burda, Harrison Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. Let's verify step by step. In *International Conference on Learning Representations*, 2024. doi: 10.48550/arXiv.2305.20050.

Zhan Ling, Yunhao Fang, Xuanlin Li, Zhiao Huang, Mingu Lee, Roland Memisevic, and Hao Su. Deductive verification of chain-of-thought reasoning. In *Advances in Neural Information Processing Systems 36*, 2023.

Pratyush Maini, Skyler Seto, He Bai, David Grangier, Yizhe Zhang, and Navdeep Jaitly. Rephrasing the web: A recipe for compute and data-efficient language modeling. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics*, 2024. doi: 10.48550/arXiv.2401.16380.

William Merrill and Ashish Sabharwal. The expressive power of transformers with chain of thought. In *International Conference on Learning Representations*, 2024.

Niklas Muennighoff, Zitong Yang, Weijia Shi, Xiang Lisa Li, Li Fei-Fei, Hannaneh Hajishirzi, Luke Zettlemoyer, Percy Liang, Emmanuel Candès, and Tatsunori Hashimoto. s1: Simple test-time scaling. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, 2025. doi: 10.48550/arXiv.2501.19393.

Maxwell Nye, Anders Johan Andreassen, Guy Gur-Ari, Henryk Michalewski, Jacob Austin, David Bieber, David Dohan, Aitor Lewkowycz, Maarten Bosma, David Luan, Charles Sutton, and Augustus Odena. Show your work: Scratchpads for intermediate computation with language models. *arXiv preprint arXiv:2112.00114*, 2021. doi: 10.48550/arXiv.2112.00114.

OLMo Team, Pete Walsh, Luca Soldaini, Dirk Groeneveld, Kyle Lo, Shane Arora, Akshita Bhagia, Yuling Gu, Shengyi Huang, Matt Jordan, et al. 2 OLMo 2 furious. In *Conference on Language Modeling*, 2025. doi: 10.48550/arXiv.2501.00656.

Liangming Pan, Alon Albalak, Xinyi Wang, and William Yang Wang. Logic-LM: Empowering large language models with symbolic solvers for faithful logical reasoning. In *Findings of the Association for Computational Linguistics: EMNLP 2023*, 2023. doi: 10.18653/v1/2023.findings-emnlp.248.

Elija Perrier. Typed chain-of-thought: A curry-howard framework for verifying LLM reasoning. *arXiv preprint arXiv:2510.01069*, 2025. doi: 10.48550/arXiv.2510.01069.

Michael Pieler, Alexander Nguyen, Akshita Bhagia, Nikhil Kandpal, Loubna Ben Allal, Dirk Groeneveld, and Luca Soldaini. BeyondWeb: Lessons from scaling synthetic data for trillion-scale pretraining. *arXiv preprint arXiv:2508.10975*, 2025. doi: 10.48550/arXiv.2508.10975.

Stanislas Polu and Ilya Sutskever. Generative language modeling for automated theorem proving. *arXiv preprint arXiv:2009.03393*, 2020. doi: 10.48550/arXiv.2009.03393.

Ben Prystawski, Michael Y. Li, and Noah D. Goodman. Why think step by step? reasoning emerges from the locality of experience. In *Advances in Neural Information Processing Systems 36*, 2023.

Abulhair Saparov and He He. Language models are greedy reasoners: A systematic formal analysis of chain-of-thought. In *International Conference on Learning Representations*, 2023.

Amrith Setlur, Nived Rajaraman, Sergey Levine, and Aviral Kumar. Scaling test-time compute without verification or RL is suboptimal. In *Proceedings of the 42nd International Conference on Machine Learning*, 2025. doi: 10.48550/arXiv.2502.12118.

Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo. DeepSeekMath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024. doi: 10.48550/arXiv.2402.03300.

Charlie Snell, Jaehoon Lee, Kelvin Xu, and Aviral Kumar. Scaling LLM test-time compute optimally can be more effective than scaling parameters for reasoning. In *International Conference on Learning Representations*, 2025. doi: 10.48550/arXiv.2408.03314.

Peiyang Song, Kaiyu Yang, and Anima Anandkumar. Lean copilot: Large language models as copilots for theorem proving in lean. *arXiv preprint arXiv:2404.12534*, 2024. doi: 10.48550/arXiv.2404.12534.

Zayne Sprague, Fangcong Yin, Juan Diego Rodriguez, Dongwei Jiang, Manya Wadhwa, Prasann Singhal, Xinyu Zhao, Xi Ye, Kyle Mahowald, and Greg Durrett. To CoT or not to CoT? chain-of-thought helps mainly on math and symbolic reasoning. In *International Conference on Learning Representations*, 2025. doi: 10.48550/arXiv.2409.12183.

Oyvind Tafjord, Bhavana Dalvi, and Peter Clark. ProofWriter: Generating implications, proofs, and abductive statements over natural language. In *Findings of the Association for Computational Linguistics: ACL-IJCNLP 2021*, 2021. doi: 10.18653/v1/2021.findings-acl.317.

Miles Turpin, Julian Michael, Ethan Perez, and Samuel R. Bowman. Language models don't always say what they think: Unfaithful explanations in chain-of-thought prompting. In *Advances in Neural Information Processing Systems 36*, 2023. doi: 10.48550/arXiv.2305.04388.

Pablo Villalobos, Anson Ho, Jaime Sevilla, Tamay Besiroglu, Lennart Heim, and Marius Hobbhahn. Position: Will we run out of data? limits of LLM scaling based on human-generated data. In *Proceedings of the 41st International Conference on Machine Learning*, 2024. doi: 10.48550/arXiv.2211.04325.

Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc V. Le, Ed H. Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. Self-consistency improves chain of thought reasoning in language models. In *International Conference on Learning Representations*, 2023. doi: 10.48550/arXiv.2203.11171.

Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed H. Chi, Quoc V. Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems 35*, 2022. doi: 10.48550/arXiv.2201.11903.

Yuhuai Wu, Albert Q. Jiang, Wenda Li, Markus N. Rabe, Charles Staats, Mateja Jamnik, and Christian Szegedy. Autoformalization with large language models. In *Advances in Neural Information Processing Systems 35*, 2022.

An Yang, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, et al. Qwen2.5 technical report. *arXiv preprint arXiv:2412.15115*, 2024. doi: 10.48550/arXiv.2412.15115.

Kaiyu Yang, Aidan Swope, Alex Gu, Rahul Chalamala, Peiyang Song, Shixing Yu, Saad Godil, Ryan Prenger, and Anima Anandkumar. LeanDojo: Theorem proving with retrieval-augmented language models. In *Advances in Neural Information Processing Systems 36 Datasets and Benchmarks Track*, 2023.

Zitong Yang, Neil Band, Shuangping Li, Emmanuel Candès, and Tatsunori Hashimoto. Synthetic continued pretraining. In *International Conference on Learning Representations*, 2025. doi: 10.48550/arXiv.2409.07431.

Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Thomas L. Griffiths, Yuan Cao, and Karthik Narasimhan. Tree of thoughts: Deliberate problem solving with large language models. In *Advances in Neural Information Processing Systems 36*, 2023.

Tian Ye, Zicheng Xu, Yuanzhi Li, and Zeyuan Allen-Zhu. Physics of language models: Part 2.1, grade-school math and the hidden reasoning process. In *International Conference on Learning Representations*, 2025a. doi: 10.48550/arXiv.2407.20311.

Tian Ye, Zicheng Xu, Yuanzhi Li, and Zeyuan Allen-Zhu. Physics of language models: Part 2.2, how to learn from mistakes on grade-school math problems. In *International Conference on Learning Representations*, 2025b. doi: 10.48550/arXiv.2408.16293.

Yang Yue, Zhiqi Chen, Rui Lu, Andrew Zhao, Zhaokai Wang, Yang Yue, Shiji Song, and Gao Huang. Does reinforcement learning really incentivize reasoning capacity in LLMs beyond the base model? In *Advances in Neural Information Processing Systems 39*, 2025. doi: 10.48550/arXiv.2504.13837.

Eric Zelikman, Yuhuai Wu, Jesse Mu, and Noah D. Goodman. STaR: Bootstrapping reasoning with reasoning. In *Advances in Neural Information Processing Systems 35*, 2022.

Honghua Zhang, Liunian Harold Li, Tao Meng, Kai-Wei Chang, and Guy Van den Broeck. On the paradox of learning to reason from data. In *Proceedings of the 32nd International Joint Conference on Artificial Intelligence*, 2023.

Denny Zhou, Nathanael Schärli, Le Hou, Jason Wei, Nathan Scales, Xuezhi Wang, Dale Schuurmans, Claire Cui, Olivier Bousquet, Quoc V. Le, and Ed H. Chi. Least-to-most prompting enables complex reasoning in large language models. In *International Conference on Learning Representations*, 2023.

A Answer-Uniqueness Validation of the Generator

Every metric in this paper compares a generated answer against a single labelled answer, which is only meaningful if the labelled answer is the sole one derivable from the premises. The generator enforces this by construction and the property is verified independently.

Each example introduces fresh constants `c0` through `c{depth}`, so no constant is reused across layers and no two branches can derive the same atom through coincidental reuse of a state predicate. A preflight check computes the full Horn closure of 1,000 training instances and 2,000 evaluation instances with a solver independent of the generator, and records the number of candidate final states derivable in each. The unique-answer rate is 1.0, the maximum derived-candidate count is 1, there are no generation failures, and answer positions are balanced across the four candidates.

Token headroom is verified over the same instances. Evaluation uses a 16,384-token context with a 7,168-new-token generation cap applied identically to both trace languages. Across every validation row and every depth, the minimum measured headroom is 494 tokens for generation and 2,788 tokens for the full context, so neither trace language is truncated at any depth reported in this paper.

B Estimating Training-Run Variance

The bootstrap intervals in Table 4 resample evaluation documents and therefore capture evaluation sampling error alone. Variability attributable to training was estimated separately in an auxiliary experiment on the same evaluation suite.

Five instruction-tuning conditions, differing only in which synthetic mixture was blended into the instruction data, were each trained twice from the same base checkpoint under different seeds, and all ten resulting models were evaluated with the identical protocol of Appendix K. For each condition and each task, the absolute difference between the two seeds gives one observation of run-to-run variability. Over the fifty resulting condition and task pairs the mean absolute difference is 0.023, the median is 0.011 and the maximum is 0.135. Treating the two runs of a condition as independent draws implies a per-run standard deviation near 0.019, and therefore a standard deviation near 0.027 for a difference between two independently seeded conditions.

Combining that term with the evaluation term gives a standard deviation near 0.033 for the contrast between the two trace conditions in Section 6.4. The largest seed differences fall on the same task family that carries the trace-language contrast, which is why the contrast is reported in Section 6.6 as consistent with the controlled study and not as independently established.

C Task Details

BranchProof samples a target proof path of controlled depth and adds distractor branches that are locally coherent but do not support the queried answer. Shortcut variants add training-only marker correlations that identify the active branch with a chosen probability. Evaluation removes those markers.

AttrCon samples objects, slots, values, and propagation constraints. The formal trace names slot/value relations directly. The natural trace describes the same propagation in controlled prose. This changes the symbolic surface while preserving the paired-rendering principle.

D Training and Evaluation Details

All main SFT runs use materialized datasets so that the formal and natural conditions see the same prompts and answers. Each train-depth subset contains 50k examples, and the online validation subset contains 256 examples. Main runs train for 10k optimizer steps with LoRA adapters on `q_proj`, `k_proj`, `v_proj`, `o_proj`, `up_proj`, `down_proj`, and `gate_proj`. The LoRA rank is 16, alpha is 32, dropout is 0.05, the optimizer learning rate is 10^{-4}, the per-device batch size is 1, gradient accumulation is 1 unless an ablation states otherwise, and training uses BF16 with gradient checkpointing.

Post-hoc evaluation uses vLLM with top-$p = 0.95$, temperature 1.0, 16 sampled generations per prompt, and pass@k values $\{1, 2, 4, 8, 16\}$. The explicit depth grid is $\{1, 2, 5, 10, 12, 15, 18, 20, 25, 30, 35, 40, 45, 50\}$. For each seed and depth we evaluate 32 prompts, so a train-1-to-25 BRANCHPROOF long-depth band contains $5 \times 32 = 160$ prompts per seed. Reported uncertainty is seed-level standard deviation, not a confidence interval.

E TRACE GRAMMAR AND VALIDATION

Formal traces contain four blocks: constants, predicates, premises, and proof lines, followed by a conclusion and answer. A proof line contains a formula and a rule justification such as premise retrieval or implication elimination. The formal checker parses the generated blocks and verifies that each proof line is licensed by earlier lines and premises. Natural-language traces use controlled sentences for the same premises and proof steps. When possible, a task-specific translator maps those sentences back into the formal checker. Because translation coverage is task-specific, final-answer correctness is the primary cross-task metric and translated validity is reported as a diagnostic.

The operational BRANCHPROOF formal surface is:

```
<formal>
<constants> CONST_DEF+ </constants>
<predicates> PRED_DEF+ </predicates>
<premises> FORMULA+ </premises>
<proof> PROOF_LINE+ </proof>
<conclusion> ATOM </conclusion>
<answer> LABEL </answer>

CONST_DEF  ::= CONST "=" LABEL
PRED_DEF   ::= PRED "x:" "x is" LABEL
ATOM       ::= PRED CONST
FORMULA    ::= ATOM | ATOM "&" ATOM "->" ATOM | ATOM "->" ATOM
PROOF_LINE ::= FORMULA "; R," REF
             | ATOM "; ->E," REF "," REF
             | ATOM "&" ATOM "; &I," REF "," REF
```

The ATTRCON surface uses the same block structure but replaces unary predicate atoms with object-slot-value relations. In both tasks, references must point to earlier proof lines or listed premises. Generated lines cannot justify future steps.

F GENERATOR AND EVALUATION MECHANICS

The BRANCHPROOF generator samples a target path, assigns labels to constants and predicates, then adds distractor branches that remain locally coherent but do not derive the queried conclusion. In pseudocode:

```
sample target answer label
sample initial object state
for depth step in 1..D:
    sample active branch rule(s)
    derive next object state
    add coherent distractor rules for inactive branches
render one prompt
render the same latent proof as formal and natural-language targets
```

The ATTRCON generator follows the same paired-rendering principle with objects, slots, values, and propagation constraints. The natural-language target is not free-form prose. It is controlled text produced from the same latent derivation.

Answer extraction first searches for the final `<answer>` field. If no well-formed final answer is present, the generation is counted incorrect even when earlier text contains the gold label. Formal

validation then parses block boundaries, declarations, formulas, proof-line references, and rule applications. Common validation failures are malformed block structure, undeclared symbols, references to unavailable lines, applying implication elimination to the wrong antecedent, and stopping before the queried conclusion. Natural-language validation adds one more possible failure mode: the controlled sentence must translate back into a formal step before the proof checker can evaluate it.

G PAIRED TARGET EXAMPLES

The following minimal examples show the paired target format used throughout the experiments. The prompt and answer are identical within each pair. Only the target trace substrate changes. Full experimental examples include more distractor premises and longer derivations.

BRANCHPROOF example 1.

```
Prompt:
a is ruby. a is north.
If a is north and a is ruby, then b is olive.
What is b?
```

Formal target

```
<formal>
<constants> a = a b = b </constants>
<predicates> Rx: x is ruby Nx: x is north
Ox: x is olive </predicates>
<premises> Ra Na Na & Ra -> Ob </premises>
<proof> Ra ; R,1
Na ; R,2
Na & Ra ; &I,5,4
Ob ; ->E,3,6 </proof>
<conclusion> Ob </conclusion>
<answer> olive </answer>
```

Natural-language target

```
<think>
a is ruby. a is north.
Since a is north and a is ruby, b is olive.
Therefore, b is olive.
</think>
<answer> olive </answer>
```

BRANCHPROOF example 2.

```
Prompt:
a is cobalt. a is east.
If a is east and a is cobalt, then b is maple.
If b is maple, then b is south.
If b is south and b is maple, then c is cedar.
What is c?
```

Formal target

```
<formal>
<constants> a = a b = b c = c </constants>
<predicates> Cx: x is cobalt Ex: x is east
Mx: x is maple Sx: x is south Dx: x is cedar </predicates>
<premises> Ca Ea
Ea & Ca -> Mb
Mb -> Sb
Sb & Mb -> Dc </premises>
<proof> Ca ; R,1
Ea ; R,2
Ea & Ca ; &I,7,6
Mb ; ->E,3,8
Sb ; ->E,4,9
Sb & Mb ; &I,10,9
Dc ; ->E,5,11 </proof>
<conclusion> Dc </conclusion>
<answer> cedar </answer>
```

Natural-language target

```
<think>
a is cobalt. a is east.
Since a is east and a is cobalt, b is maple.
Since b is maple, b is south.
Since b is south and b is maple, c is cedar.
Therefore, c is cedar.
</think>
<answer> cedar </answer>
```

ATTRCON example 1.

```
Prompt:
s0 has value red. s1 has value blue.
If s0 is red and s1 is blue, then s2 is green.
```

```
Which values fill all constrained slots?
```

Formal target

```
<formal>
<constants> s0 = attribute slot 0 s1 = attribute slot 1
s2 = attribute slot 2
red = attribute value red
blue = attribute value blue
green = attribute value green </constants>
<predicates> Value(x,y): slot x has value y
Constraint(x,y,z,w,u,v): values y and w at slots x and z
jointly force value v at slot u </predicates>
<premises> Value(s0,red) Value(s1,blue)
Constraint(s0,red,s1,blue,s2,green)
Value(s0,red) & Value(s1,blue) &
Constraint(s0,red,s1,blue,s2,green)
-> Value(s2,green) </premises>
<proof> Value(s0,red) ; R,1
Value(s1,blue) ; R,2
Constraint(s0,red,s1,blue,s2,green) ; R,3
Value(s0,red) & Value(s1,blue) ; &I,5,6
Value(s0,red) & Value(s1,blue) &
Constraint(s0,red,s1,blue,s2,green) ; &I,8,7
Value(s2,green) ; ->E,4,9 </proof>
<answer> red-blue-green </answer>
```

Natural-language target

```
<think>
s0 has value red.
s1 has value blue.
The applicable joint constraint maps s0=red and s1=blue
to s2=green.
Combine the two prerequisite slot values.
Therefore s2 has value green.
</think>
<answer> red-blue-green </answer>
```

ATTRCON example 2.

```
Prompt:
s0 has value amber. s1 has value ivory.
If s0 is amber and s1 is ivory, then s2 is olive.
If s0 is amber and s2 is olive, then s3 is coral.
Which values fill all constrained slots?
```

Formal target

```
<formal>
<constants> s0 = attribute slot 0 s1 = attribute slot 1
s2 = attribute slot 2 s3 = attribute slot 3
amber = attribute value amber
ivory = attribute value ivory
olive = attribute value olive
coral = attribute value coral </constants>
<predicates> Value(x,y): slot x has value y
Constraint(x,y,z,w,u,v): values y and w at slots x and z
jointly force value v at slot u </predicates>
<premises> Value(s0,amber) Value(s1,ivory)
Constraint(s0,amber,s1,ivory,s2,olive)
Value(s0,amber) & Value(s1,ivory) &
Constraint(s0,amber,s1,ivory,s2,olive) -> Value(s2,olive)
Constraint(s0,amber,s2,olive,s3,coral)
Value(s0,amber) & Value(s2,olive) &
Constraint(s0,amber,s2,olive,s3,coral)
-> Value(s3,coral) </premises>
<proof> Value(s0,amber) ; R,1
Value(s1,ivory) ; R,2
Constraint(s0,amber,s1,ivory,s2,olive) ; R,3
Value(s0,amber) & Value(s1,ivory) ; &I,7,8
Value(s0,amber) & Value(s1,ivory) &
Constraint(s0,amber,s1,ivory,s2,olive) ; &I,10,9
Value(s2,olive) ; ->E,4,11
Constraint(s0,amber,s2,olive,s3,coral) ; R,5
Value(s0,amber) & Value(s2,olive) ; &I,7,12
Value(s0,amber) & Value(s2,olive) &
Constraint(s0,amber,s2,olive,s3,coral) ; &I,14,13
Value(s3,coral) ; ->E,6,15 </proof>
<answer> amber-ivory-olive-coral </answer>
```

Natural-language target

```
<think>
s0 has value amber.
s1 has value ivory.
The applicable joint constraint maps s0=amber and s1=ivory
to s2=olive.
Combine the two prerequisite slot values.
Therefore s2 has value olive.
The applicable joint constraint maps s0=amber and s2=olive
to s3=coral.
Combine the two prerequisite slot values.
Therefore s3 has value coral.
</think>
<answer> amber-ivory-olive-coral </answer>
```

H METRIC DEFINITIONS

Answer correctness is computed after extracting the final answer field from the generation. Formal joint validity requires both answer correctness and a mechanically valid formal trace. Natural-language joint validity requires answer correctness and successful translation of the generated natural-

Table 5: Rendered document lengths under the Qwen2.5-7B tokenizer, 72,000 documents per corpus, and packing statistics at sequence length 8,192.

corpus	p50	mean	p99	max	windows	real tokens	pack. eff.
Logic	3,749	3,856	7,673	7,727	35,492	277,732,923	0.9551
Natural	3,819	3,907	7,744	7,851	35,979	281,372,725	0.9545
Condensed	1,450	1,500	3,007	3,014	13,311	108,099,330	0.9912
LongDoc	3,712	3,832	n/a	7,750	n/a	n/a	n/a

language trace into a valid formal proof. Because the natural-language translator is task-specific, answer correctness is the primary cross-task metric.

I Length and Token-Budget Controls

Formal and natural renderings of the same latent proof differ in length, which raises the possibility that the controlled comparison measures the quantity of supervision. Three controls bound that explanation.

Generation budgets are identical by construction. Both conditions receive a 7,168-token generation limit inside a 16,384-token window. A tokenizer audit over the full evaluation set confirms that every gold prompt and every gold target fits inside that limit in both renderings, so neither condition is truncated at generation time and neither is advantaged by a looser budget.

Supervised token counts are equated in a matched-token training run, in which the number of target tokens per condition is held equal by subsampling the longer condition. The ordering of the two conditions is unchanged.

The transfer study of Section 6 supplies a third control at a different scale. The **LongDoc** condition matches the token-length histogram of the formal traces to within one percent at the median while carrying no deep deduction, and it does not reproduce the gain of the trace conditions on the external benchmark. Length alone therefore does not account for the effect in either setting.

J Boundary Condition Details

Capacity. Single-modality training at three scales gives out-of-distribution answer pass@1 of 69.9 ± 11.1 formal against 74.9 ± 7.0 natural on Qwen2.5-7B, and 63.9 ± 5.1 formal against 98.9 ± 0.8 natural on OLMo-3-32B. Formal answer pass@16 at 32B is 98.8 ± 0.9, so the deficit at 32B is a deficit of probability mass and not of coverage.

Mode conditioning. Concatenating both renderings into a single target yields 0.5 ± 0.4 percent out-of-distribution pass@1 with the formal rendering first and 0.2 ± 0.1 percent with the natural rendering first. Mode-conditioned training with an output-mode tag reaches 85.0 ± 8.7 answer pass@1 in formal mode at 32B and 77.2 ± 11.7 in natural mode. Raw generations show clean separation between the modes.

Shortcuts. A spurious schema marker planted on 80 percent of training examples and removed at test time gives 43.1 ± 7.7 out-of-distribution pass@1 for formal against 78.5 ± 14.9 for natural. Additional marker types and injection rates were measured and are not reported here.

Scale floor. From-scratch training at 50M to 200M parameters on the same corpora gives out-of-distribution joint pass@k of 0.0 for all $k \leq 8$ at every size in both conditions.

K Midtraining Configuration

Optimisation. Every condition trains Qwen2.5-7B for 2,385 steps at sequence length 8,192 with a global batch of 128 sequences, formed as micro-batch 2 with gradient accumulation 32 over eight

A100-80 devices. The resulting budget is $2{,}500{,}853{,}760$ tokens per condition, chosen to equal $4{,}770 \times 128 \times 4{,}096$ so that it matches the token budget of an earlier study at half the sequence length. Learning rate is 1×10^{-5} with minimum 1×10^{-6}, 128 warmup steps, decay beginning at step 128 and a decay horizon of 18,946 steps. Checkpoints are written every 250 steps. Measured throughput is approximately 36.0 seconds per iteration and 29,000 tokens per second, at 191 TFLOP/s per device. Collective-operation timeouts are set to 120 minutes. A run exceeds the 24-hour scheduling limit and is completed as two serialized allocations, with the second resuming from the newest complete checkpoint.

Replaced slice. The 72,000 source examples are generated at seed 20260907 for the reinforcement-learning study and at seed 20260830 for the midtraining study, with depths 1 to 25 sampled round-robin, branching factor 4, distractor ratio 0.5 and the `hard_fsa_schema` variant. Seeds are disjoint from those used for evaluation corpora and for sampling. A per-condition document weight is solved so that the realised synthetic share of loss tokens is exactly ten percent after padding is accounted for, which is necessary because the three renderings have different packing efficiencies.

Packing. Documents are packed by a document-preserving packer at sequence length 8,192 under a fixed shuffle seed of 42. The packer never splits a document. A document exceeding one window is excluded and counted as overlength. Tail padding uses a dedicated padding token that is masked out of the loss. Overlength and split counts are zero for every condition, reported in Table 5. Before training begins, a decoded-batch audit reconstructs real loader windows and verifies that no split document is present, that the padding token is masked, and that the realised replacement share matches the target. Training refuses to start if any gate fails.

Instruction tuning. Each midtrained checkpoint is converted to the Hugging Face format and instruction-tuned on 100,000 examples of a public instruction corpus at a pinned revision, for one epoch at learning rate 5×10^{-6} with warmup ratio 0.03, maximum sequence length 4,096, per-device batch size 1, and seed 3407. Training is full-parameter under fully sharded data parallelism with gradient checkpointing. Only the final checkpoint is written.

Evaluation. Greedy evaluation uses a standard harness with a paged-attention inference backend in bfloat16, a maximum model length of 8,192, a chat template applied to every prompt, and per-sample logging enabled. The reported metric is exact match on the extracted answer. The external family is ProofWriter under the open-world assumption at inference depths 0, 1, 2, 3 and 5 with 500 items per depth. The near-transfer family is a held-out sample from the same generator at depths 5, 10, 15, 20 and 25 with 200 items per depth.

Sampled evaluation draws sixteen completions per item at temperature 0.8 and top-p 0.95 under a fixed seed, with a 2,048-token response limit. Prompts are replayed verbatim from the logged greedy samples, so the greedy and sampled measurements differ in the sampler alone and in nothing else. The scorer used for sampled generations imports the same extraction and matching function used by the greedy harness, and was verified to reproduce all fifteen published greedy exact-match values before any sampled number was computed. Audits require the expected item count per task, no empty generations beyond a tolerance, and no truncated prompts.

Statistics. Contrasts in Table 4 come from a paired bootstrap over evaluation documents with 4,000 resamples, resampling document indices once per replicate and applying the same indices to every condition and depth. Reported intervals are the 2.5th and 97.5th percentiles of the replicate distribution. Training-run variance is estimated separately from an earlier campaign on the same evaluation suite in which five conditions were each trained under two instruction-tuning seeds, giving fifty condition and task pairs.

L Extended Testbed Description

Each example begins with a latent directed acyclic proof. The generator emits a prompt containing ground facts, rules, a query and distractor branches, then traverses the supporting path to produce the target answer. A rendering function writes that path in one of two trace languages. The FORMAL rendering declares constants and predicates and applies named inference rules to numbered lines.

Table 6: Formal minus natural on the near-transfer derivation family, by inference depth and decoding rule. Positive favours the formal condition. The sign of the contrast follows the decoding rule, with no clear depth trend.

decoding	d5	d10	d15	d20	d25
greedy	-0.175	-0.165	-0.055	-0.040	-0.105
pass@16	$+0.035$	$+0.070$	$+0.140$	$+0.040$	-0.035
maj@16	-0.140	-0.030	-0.005	$+0.025$	-0.070

The NATURAL rendering expresses the same substitutions and conclusions, line for line, in controlled sentences. Both targets end with the same answer field, and the prompt is natural language in both conditions.

BRANCHPROOF tests composition across a chain of Horn-style inferences in which every step offers four locally plausible branches, exactly one of which is derivable from the current state. Every layer introduces a fresh constant. The generator computes the full Horn closure of each example and accepts it only when exactly one candidate answer is derivable, which removes the possibility that a model is rewarded for guessing among several defensible answers. ATTRCON propagates values among object attributes under generated constraints. Its vocabulary and proof construction are disjoint from BRANCHPROOF, so it provides an independent test of the same hypothesis. Appendix G gives complete paired targets, Appendices E and F specify the grammar, the generator and the validators, and Appendix A documents the answer-uniqueness audit of the final generator.

Generation parameters are held fixed across every controlled experiment reported below. The branching factor is 4, the distractor ratio is 0.5, and the schema variant is the one denoted `hard_fsa_schema` in the release. Depths are sampled round-robin so that each depth in the training range contributes an equal number of examples. Seeds are recorded per corpus and are disjoint across roles, so that no corpus used for training shares a seed with a corpus used for evaluation or for sampling.

M PER-DEPTH TRANSFER CONTRASTS